



МИНОБРАЗОВАНИЯ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий (ИТ)

Кафедра Математического обеспечения и стандартизации информационных
технологий (МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 4

по дисциплине

«Технология разработки программных приложений»

Тема: «Docker»

Выполнил студент группы ИКБО-66-23

Смирнов А.Ю.

Принял

Петренко А.А.

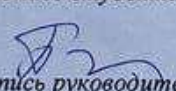
Практическая работа выполнена

«11» 04 2025 г.


(подпись студента)

«Зачтено»

«11» 04 2025 г.


(подпись руководителя)

Москва 2025

Оглавление

Ответы на вопросы.....	2
Индивидуальное задание.....	7
Вывод о проделанной работе.....	9

Ответы на вопросы

Посмотрим на имеющиеся образы: `docker images`.

```
C:\Users\theso>docker images
REPOSITORY    TAG        IMAGE ID      CREATED      SIZE
```

Рисунок 1 - Образы

Загрузим образ: `docker pull ubuntu`.

```
C:\Users\theso>docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
2726e237d1a3: Pull complete
Digest: sha256:1e622c5f073b4f6bfad6632f2616c7f59ef256e96fe78bf6a595d1dc4376ac02
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
```

Рисунок 2 – Загрузка образа

Посмотрим на имеющиеся образы ещё раз: `docker images`.

```
C:\Users\theso>docker images
REPOSITORY    TAG        IMAGE ID      CREATED      SIZE
ubuntu        latest     1e622c5f073b 30 hours ago 117MB
```

Рисунок 3 - Образы

Посмотрим список контейнеров, выполнив команду: `docker ps -a`

```
C:\Users\theso>docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
```

Рисунок 4 – Контейнеры

Посмотрим информацию о хостовой системе, выполнив команду `hostname`. Выполним её ещё один раз. Одинаковый ли результат получился при разных запусках? Да.

Попробуем выполнить то же самое в контейнерах. Выполним два раза команду `docker run ubuntu hostname`. Одинаковый ли результат получился при разных запусках? Нет.

```

C:\Users\theso>hostname
Alper

C:\Users\theso>hostname
Alper

C:\Users\theso>docker run ubuntu hostname
b51ab4e682b5

C:\Users\theso>docker run ubuntu hostname
0f9fd7eff8fd

```

Рисунок 5 – Информация о хостовой системе

Заново выполним `docker ps -a` — там должны появиться запущенные ранее контейнеры.

```

C:\Users\theso>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS              PORTS          NAMES
0f9fd7eff8fd   ubuntu   "hostname"              17 seconds ago Exited (0) 16 seconds ago           admiring_montalcini
b51ab4e682b5   ubuntu   "hostname"              21 seconds ago Exited (0) 20 seconds ago           great_fermi

```

Рисунок 6 – Запущенные ранее контейнеры

Запустим `bash` в контейнере: `docker run ubuntu bash`. Добавим флаги `-i -t` или сгруппированно `-it`: `docker run -it ubuntu bash`.

```

C:\Users\theso>docker run ubuntu bash

C:\Users\theso>docker run -it ubuntu bash
root@58dd11fe0c47:/# |

```

Рисунок 7 – Запуск баш

Загрузим образ `python` командой `docker pull python`.

```

C:\Users\theso>docker pull python
Using default tag: latest
latest: Pulling from library/python
739b86d2a778: Pull complete
23b7d26ef1d2: Pull complete
07d1b5af933d: Pull complete
171e1bee1949: Pull complete
b617a119f8a2: Pull complete
e25cca11fd29: Pull complete
1eb98adba0eb: Pull complete
Digest: sha256:9819e5616923079cc16af4a93d4be92c0c487c6e02fd9027220381f3e125d64a
Status: Downloaded newer image for python:latest
docker.io/library/python:latest

```

Рисунок 8 – Загрузка образа

Запустим встроенный в Python модуль веб-сервера из корня контейнера, чтобы отобразить содержание контейнера. `docker run -it python python -m http.server`.

Пробросим порт 8000: `docker run -it -p8000:8000 python python -m http.server` — теперь по адресу `http://0.0.0.0: 8000/`. Для того, чтобы доступный в контейнере на порту 8000 веб-сайт в хостовой системе открывался на порту 8888, необходимо указать флаг `-p 8888:8000`: `docker run -it -p8888:8000 python python -m http.server`. Завершим работу веб-сервера, нажав комбинацию клавиш `Ctrl+C`.

```
C:\Users\theso>docker run -it python python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

```
C:\Users\theso>docker run -it python python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
^C
Keyboard interrupt received, exiting.

C:\Users\theso>docker run -it -p 8888:8000 python python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
^C
Keyboard interrupt received, exiting.
```

Рисунок 9 – Работа с веб-сервером

Для того, чтобы запустить контейнер в фоне, нужно добавить флаг `-d/--detach`. Также определим имя контейнеру, добавив флаг `--name`.

`docker run -p8000:8000 --name pyserver -d python python -m http.server`

```
C:\Users\theso>docker run -p8000:8000 --name pyserver -d python python -m http.server
f4fab5bd8547be7e6c883018741cf492908e34361787437166c73b2f95caf413
```

Рисунок 10 – Запуск контейнера в фоне

Для того, чтобы остановить выполнение контейнера, существует команда `docker stop pyserver`.

Для остановки и удаления контейнера можно воспользоваться командой `docker rm -f pyserver` вместо выполнения двух отдельных команд `stop` и `rm`.

```
C:\Users\theso>docker stop f4fab5b
f4fab5b

C:\Users\theso>docker rm f4fab5b
f4fab5b
```

Рисунок 11 – Остановка и удаление контейнера

Для того, чтобы контейнер удалялся после завершения работы, нужно указать флаг `--rm` при его запуске

```
docker run --rm -p8000:8000 --name pyserver -d python python -m http.server
```

```
C:\Users\theso>docker run --rm -p8000:8000 --name pyserver -d python python -m http.server
1f9a3bf2e18cca83e066dc16343dca9d1672cc676c2361b6115b50bf4f802959
```

Рисунок 12 – Удаление контейнера по завершении

Запустим контейнер, в котором веб-сервер будет отдавать содержимое директории `/mnt`:

```
docker run -p8000:8000 --name pyserver --rm -d python python -m http.server -d
/mnt
```

Что значат остальные флаги запуска? Где здесь команда, которая выполнится в контейнере? `-p` – указание портов, `--name` – название контейнера, `--rm` – удаление контейнера по завершении, `-d` – запуск в фоновом режиме, `-m` – запуск сервера

Для того, чтобы попасть в уже запущенный контейнер, существует команда `docker exec -it pyserver bash` — вы попадёте в оболочку `bash` в контейнере.

```
C:\Users\theso>docker run -p8000:8000 --name pyserver --rm -d python python -m http.server -d /mnt
85fb706ba73f49c5371d1653da313c9963d170c98740719d09017070df6aaf0a

C:\Users\theso>docker exec -it pyserver bash
root@85fb706ba73f:/# |
```

Рисунок 13 – Работа с контейнером

Для передачи переменных окружения внутрь контейнера используется ключ `-e`.

```
C:\Users\theso>docker run -it --rm -e MIREA="ONE LOVE" ubuntu env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=6ef8497387fd
TERM=xterm
MIREA=ONE LOVE
HOME=/root
```

Рисунок 14 – Работа с переменными окружения

```
FROM ubuntu :20.04
RUN apt update \
    && apt install -y python3 fortune \
    && cd /usr/bin \
    && ln -s python3 python
RUN /usr/games/fortune > /mnt/greeting-while-building.txt
ADD ./data /mnt/data
EXPOSE 80
CMD ["python", "-m", "http.server", "-d", "/mnt/", "80"]
```

Рисунок 15 – Dockerfile

```
C:\Users\theso\Desktop\trpp4>docker build -t mycoolimage .
[+] Building 27.9s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 319B
=> [internal] load metadata for docker.io/library/ubuntu:20.04
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/ubuntu:20.04@sha256:8feb4d8ca5354def3d8fce243717141ce31e2c428701f6682bd2fafa15388214
=> => resolve docker.io/library/ubuntu:20.04@sha256:8feb4d8ca5354def3d8fce243717141ce31e2c428701f6682bd2fafa15388214
=> => sha256:13b7e939469f6d3575a320709035c6acf6f5485a76abcf03d1b92a64c09c2476 27.51MB / 27.51MB
=> => extracting sha256:13b7e939469f6d3575a320709035c6acf6f5485a76abcf03d1b92a64c09c2476
=> [internal] load build context
=> => transferring context: 82B
=> [2/4] RUN apt update && apt install -y python3 fortune && cd /usr/bin && ln -s python3 python
=> [3/4] RUN /usr/games/fortune > /mnt/greeting-while-building.txt
=> [4/4] ADD ./data /mnt/data
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:e29371d0f2bd06b4fb320ca8a871b88158e4baee1107fc5f04e1fa22960fabe3
=> => exporting config sha256:f61dafc90e9cec38a51aaef02e4baf94a0d526253691244769da44032881fd07
=> => exporting attestation manifest sha256:8999e96672f07803b7034859e824ea2c1dd3e7aefebbb41b68b5d9fee2802e4aa
=> => exporting manifest list sha256:8b05d8c91e9320486668b0ce2841cc2a0871695d49686d5e53de924d93365e0b
=> => naming to docker.io/library/mycoolimage:latest
=> => unpacking to docker.io/library/mycoolimage:latest
```

Рисунок 16 – Сборка проекта

```
C:\Users\theso\Desktop\trpp4>docker run --rm -it -p8099:80 mycoolimage
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
172.17.0.1 - - [09/Apr/2025 17:53:28] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [09/Apr/2025 17:53:28] code 404, message File not found
172.17.0.1 - - [09/Apr/2025 17:53:28] "GET /favicon.ico HTTP/1.1" 404 -
172.17.0.1 - - [09/Apr/2025 17:53:30] "GET /data/ HTTP/1.1" 200 -
172.17.0.1 - - [09/Apr/2025 17:53:31] "GET /data/test.txt HTTP/1.1" 200 -
172.17.0.1 - - [09/Apr/2025 17:53:33] "GET /greeting-while-building.txt HTTP/1.1" 200 -
```



Рисунок 17 - Запуск

Индивидуальное задание.

Напишем Dockerfile, соберём образ, запустим контейнер.

Для монтирования создадим директорию data и в ней файл student.txt, содержащий ФИО, название группы и номер варианта.

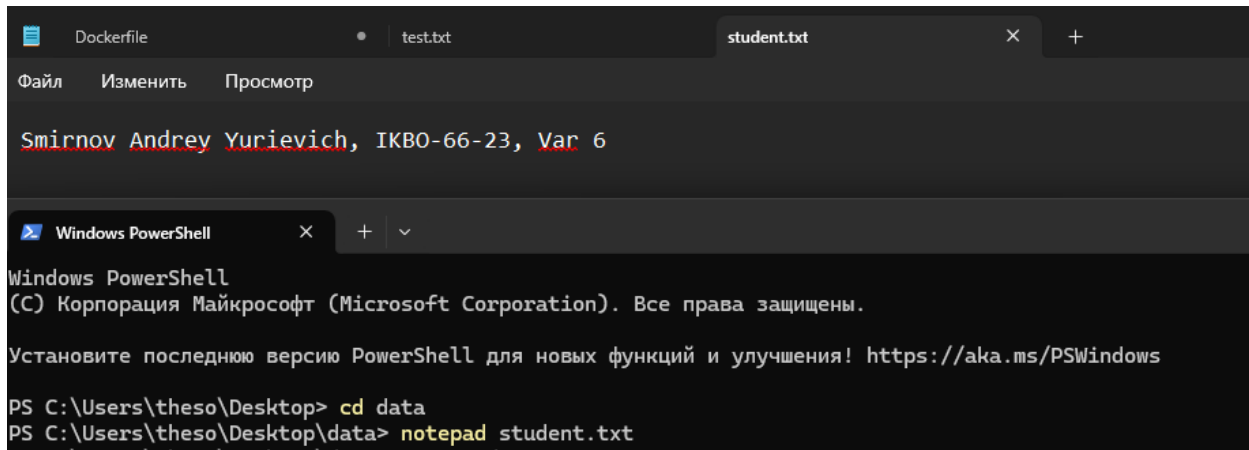


Рисунок 1 – Создание файла

Для установки пакетов используем команду `apt install -y название-пакета`. Используем базовый образ `ubuntu:20.04`, примонтируем директорию `data` в директорию `/mnt/files/` в контейнере. Запустим веб-сервер, отображающий содержимое `/mnt/files`, в хостовой системе должен открываться на порту 8809. Установим пакет `mysql-client`.

```
PS C:\Users\theso\Desktop\data> cat student.txt
Smirnov Andrey Yurievich, IKBO-66-23, Var 6
```

Рисунок 2 – Содержание файла student.txt

```
FROM ubuntu:20.04
RUN apt update && apt install -y patch python3
RUN mkdir -p /mnt/files
COPY student.txt /mnt/files/student.txt
EXPOSE 80
CMD ["python3" , "-m" , "http.server" , "-d" , "/mnt/files/" , "80"]
```

Рисунок 3 – Содержание Dockerfile, его сборка


```

View build details: docker-desktop:///dashboard/build/desktop-linux/desktop-linux/x0qmls06ei3pz4uya6te6zko
PS C:\Users\theso\Desktop\data> docker build -t proect .
[+] Building 20.0s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 251B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:20.04    1.5s
=> [auth] library/ubuntu:pull token for registry-1.docker.io      0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> CACHED [1/4] FROM docker.io/library/ubuntu:20.04@sha256:8feb4d8ca5354def3d8fce243717141ce31e2c428701f6682bd2f 0.1s
=> => resolve docker.io/library/ubuntu:20.04@sha256:8feb4d8ca5354def3d8fce243717141ce31e2c428701f6682bd2fafa1538 0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 32B                                       0.0s
=> [2/4] RUN apt update && apt install -y patch python3          13.8s
=> [3/4] RUN mkdir -p /mnt/files                                  0.4s
=> [4/4] COPY student.txt /mnt/files/student.txt                  0.1s
=> exporting to image                                              3.8s
=> => exporting layers                                              3.0s
=> => exporting manifest sha256:8ea55eab356add028fc4bfe6b9e14de6471eac881790321435a1fa4f51247595 0.0s
=> => exporting config sha256:f687096ce4fc47ala92f652622988bc1f3ad5362fe557213001da68ca06c92ab 0.0s
=> => exporting attestation manifest sha256:34c7152342437066eba4e1face7f28d149af65d9da44942bf80177f0ebbfef08 0.0s
=> => exporting manifest list sha256:c1c191ac5be2175cfca13cd9aa5ab671002a2a629f369ffbdd4ecd5d651e6b9c 0.0s
=> => naming to docker.io/library/proect:latest                    0.0s
=> => unpacking to docker.io/library/proect:latest                  0.6s

View build details: docker-desktop:///dashboard/build/desktop-linux/desktop-linux/mlu5b5j0q94afw9ws82hx0sri

```

Рисунок 4 – Сборка Dockerfile

```

View build details: docker-desktop:///dashboard/build/desktop-linux/desktop-linux/mlu5b5j0q94afw9ws82hx0sri
PS C:\Users\theso\Desktop\data> docker run -d -p 8809:80 -v C:\Users\theso\Desktop\data:/mnt/files -it proect
43803b7fe946ffbb96de51bdac20a9cd4fd612cf810e0db33efaba9ef9b8378d
PS C:\Users\theso\Desktop\data>

```

Рисунок 5 – Запуск образа

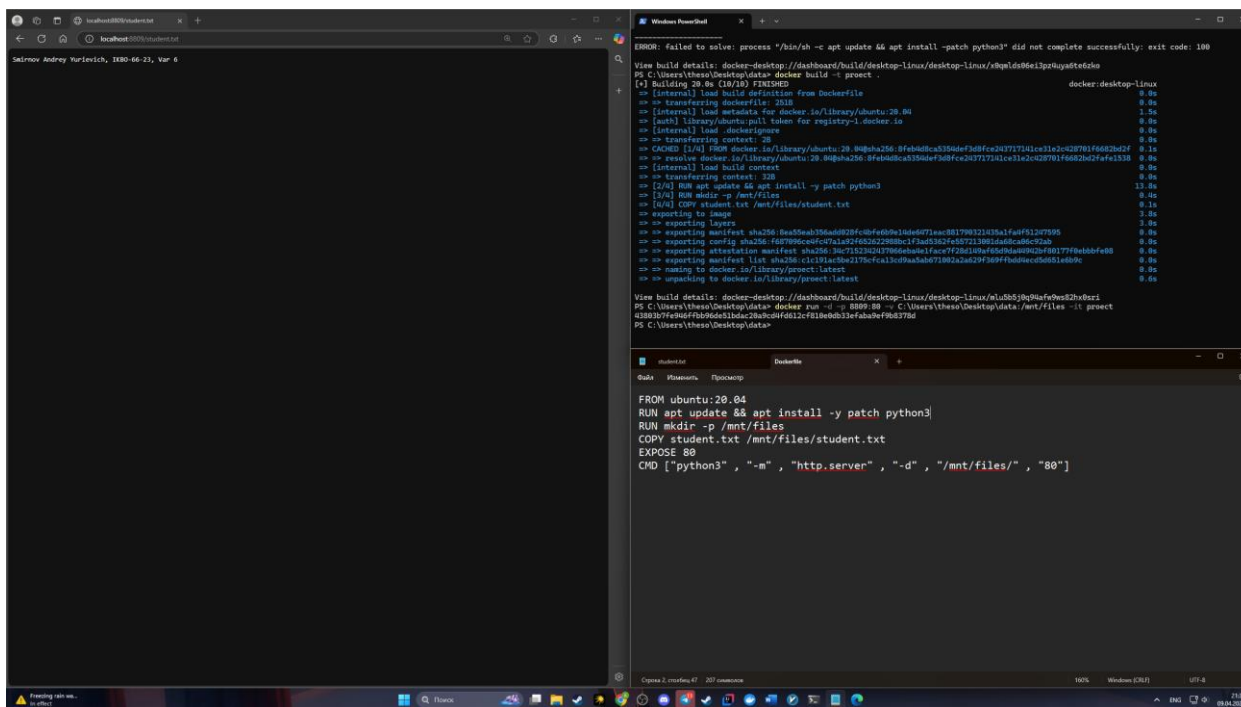


Рисунок 6 – Результат

Вывод о проделанной работе

В ходе практической работы мы получили навыки по написанию, сборке и запуске Dockerfile.